Write SOSL Queries

Learning Objectives

After completing this unit, you'll be able to:

- Describe the differences between SOSL and SOQL.
- Search for fields across multiple objects using SOSL queries.
- Execute SOSL queries by using the Query Editor in the Developer Console.

Get Started with SOSL

Salesforce Object Search Language (SOSL) is a Salesforce search language that is used to perform text searches in records. Use SOSL to search fields across multiple standard and custom object records in Salesforce. SOSL is similar to Apache Lucene.

Adding SOSL queries to Apex is simple–you can embed SOSL queries directly in your Apex code. When SOSL is embedded in Apex, it is referred to as ***inline SOSL***.

This is an example of a SOSL query that searches for accounts and contacts that have any fields with the word 'SFDC'.

```
1  List<List<SObject>> searchList = [FIND 'SFDC' IN ALL FIELDS
2                                    RETURNING Account(Name),
   Contact(FirstName,LastName)];
```

Differences and Similarities Between SOQL and SOSL

Like SOQL, SOSL allows you to search your organization's records for specific information. Unlike SOQL, which can only query one standard or custom object at a time, a single SOSL query can search all objects.

SOQL and SOSL are two separate languages with different syntax. Each language has a distinct use case:

- Use SOQL to retrieve records for a single object.
- Use SOSL to search fields across multiple objects. SOSL queries can search most text fields on an object.

Apex Basics & Database (/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Write SOSL Queries

Prerequisites

Some queries in this unit expect the org to have accounts and contacts. If you haven't created the sample data in the SOQL unit, create sample data in this unit. Otherwise, you can skip creating the sample data in this section.

1. In the Developer Console, open the Execute Anonymous window from the **Debug** menu.
2. Insert the below snippet in the window and click **Execute**.

```
// Add account and related contact
Account acct = new Account(
    Name='SFDC Computing',
    Phone='(415)555-1212',
    NumberOfEmployees=50,
    BillingCity='San Francisco');
insert acct;
// Once the account is inserted, the sObject will be
// populated with an ID.
// Get this ID.
ID acctID = acct.ID;
// Add a contact to this account.
Contact con = new Contact(
    FirstName='Carol',
    LastName='Ruiz',
    Phone='(415)555-1212',
    Department='Wingo',
    AccountId=acctID);
insert con;
// Add account with no contact
```

The Developer Console provides the Query Editor console, which enables you to run SOSL queries and view results. The Query Editor provides a quick way to inspect the database. It is a good way to test your SOSL queries before adding them to your Apex code. When you use the Query Editor, you need to supply only the SOSL statement without the Apex code that surrounds it.

Apex Basics & Database (/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)
Write SOSL Queries

Let's try running the following SOSL example:

1. In the Developer Console, click the *Query Editor* tab.
2. Copy and paste the following into the first box under Query Editor, and then click **Execute**.

```
FIND {Wingo} IN ALL FIELDS RETURNING Account(Name),
Contact(FirstName,LastName,Department)
```

All account and contact records in your org that satisfy the criteria will display in the Query Results section as rows with fields. The results are grouped in tabs for each object (account or contact). The SOSL query returns records that have fields whose values match Wingo. Based on our sample data, only one contact has a field with the value Wingo, so this contact is returned.

The search query in the Query Editor and the API must be enclosed within curly brackets (`{Wingo}` copy). In contrast, in Apex the search query is enclosed within single quotes (`'Wingo'` copy).

Basic SOSL Syntax

SOSL allows you to specify the following search criteria:

- Text expression (single word or a phrase) to search for
- Scope of fields to search
- List of objects and fields to retrieve
- Conditions for selecting rows in the source objects

This is the syntax of a basic SOSL query in Apex:

```
FIND 'SearchQuery' [IN SearchGroup] [RETURNING ObjectsAndFields]
```

SearchQuery is the text to search for (a single word or a phrase). Search terms can be grouped with logical operators (AND, OR) and parentheses. Also, search terms can include wildcard characters (*, ?). The * wildcard matches zero or more characters at the middle or end of the search term. The ? wildcard matches only one character at the middle or end of the search term.

Write SOSL Queries

Text searches are case-insensitive. For example, searching for `Customer` copy, `customer` copy, or `CUSTOMER` copy all return the same results.

SearchGroup is optional. It is the scope of the fields to search. If not specified, the default search scope is all fields. *SearchGroup* can take one of the following values.

- `ALL FIELDS` copy
- `NAME FIELDS` copy
- `EMAIL FIELDS` copy
- `PHONE FIELDS` copy
- `SIDEBAR FIELDS` copy

ObjectsAndFields is optional. It is the information to return in the search result–a list of one or more sObjects and, within each sObject, a list of one or more fields, with optional values to filter against. If not specified, the search results contain the IDs of all objects found.

Single Words and Phrases

A *SearchQuery* contains two types of text:

- **Single Word**– single word, such as `test` copy or `hello` copy. Words in the `SearchQuery` copy are delimited by spaces, punctuation, and changes from letters to digits (and vice-versa). Words are always case insensitive.
- **Phrase**– collection of words and spaces surrounded by double quotes such as `"john smith"` copy. Multiple words can be combined together with logic and grouping operators to form a more complex query.

Search Examples

To learn about how SOSL search works, let's play with different search strings and see what the output is based on our sample data. This table lists various example search strings and the SOSL search results.

The Query	This search returns all records whose fields contain both words: The and Query, in any location of the text. The order of words in the search term doesn't matter.	Account: The SFDC Query Man (Name field matched)
Wingo OR Man	This search uses the OR logical operator. It returns records with fields containing the word Wingo or records with fields containing the word Man.	Contact: Carol Ruiz, Department: 'Wingo' Account: The SFDC Query Man (Name field matched)
1212	This search returns all records whose fields contain the word 1212. Phone fields that end with -1212 are matched because 1212 is considered a word when delimited by the dash.	Account: SFDC Computing, Phone: '(415)555-1212' Contact: Carol Ruiz, Phone: '(415)555-1212'
wing*	This is a wildcard search. This search returns all records that have a field value starting with wing.	Contact: Maria Ruiz, Department: 'Wingo' Account: The SFDC Query Man, Description: 'Expert in wing technologies.'

Apex Basics & Database (/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Write SOSL Queries

SOSL Apex Example

This example shows how to run a SOSL query in Apex. First, the variable `soslFindClause` copy is assigned the search query, which consists of two terms (Wingo and SFDC) combined by the OR logical operator. The SOSL query references this local variable by preceding it with a colon, also called *binding*. The resulting SOSL query searches for Wingo or SFDC in any field. This example returns all the sample accounts because they each have a field containing one of the words. The SOSL search results are returned in a list of lists. Each list contains an array of the returned records. In this case, the list has two elements. At index 0, the list contains the array of accounts. At index 1, the list contains the array of contacts.

Apex Basics & Database (/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Write SOSL Queries

```
List<List<sObject>> searchList = [FIND :soslFindClause IN ALL
FIELDS
                    RETURNING
Account(Name), Contact(FirstName,LastName,Department)];
Account[] searchAccounts = (Account[])searchList[0];
Contact[] searchContacts = (Contact[])searchList[1];
System.debug('Found the following accounts.');
for (Account a : searchAccounts) {
    System.debug(a.Name);
}
System.debug('Found the following contacts.');
for (Contact c : searchContacts) {
    System.debug(c.LastName + ', ' + c.FirstName);
}
```

Tell Me More...

You can filter, reorder, and limit the returned results of a SOSL query. Because SOSL queries can return multiple sObjects, those filters are applied within each sObject inside the RETURNING clause.

You can filter SOSL results by adding conditions in the WHERE clause for an object. For example, this results in only accounts whose industry is Apparel to be returned: `RETURNING Account(Name, Industry WHERE Industry='Apparel')` copy.

Likewise, ordering results for one sObject is supported by adding ORDER BY for an object. For example this causes the returned accounts to be ordered by the Name field: `RETURNING Account(Name, Industry ORDER BY Name)` copy.

The number of returned records can be limited to a subset of records. This example limits the returned accounts to 10 only: `RETURNING Account(Name, Industry LIMIT 10)` copy.

Resources

- SOQL and SOSL Reference (https://developer.salesforce.com/docs/atlas.en-us.soql_sosl.meta/soql_sosl/)

Apex Basics & Database (/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Write SOSL Queries

Create an Apex class that returns both contacts and leads based on a parameter.

To pass this challenge, create an Apex class that returns both contacts and leads that have first or last name matching the incoming parameter.

- The Apex class must be called **ContactAndLeadSearch** and be in the public scope
- The Apex class must have a public static method called **searchContactsAndLeads**
  - The method must accept an incoming string as a parameter
  - The method should then find any contact or lead whose first or last name match the string
  - The method should finally use a return type of **List<List< sObject>>**
- **NOTE:** Because SOSL indexes data for searching, you must create a Contact record and Lead record before checking this challenge. Both records must have the last name **Smith**. The challenge uses these records for the SOSL search

Choose hands-on org

Koushik nelluri Salesforce CRM
Created on 5/9/2026

Launch

Check Challenge to Earn 500 Points

Report Illegal Content (EU)

Apex Basics & Database (/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Write SOSL Queries